

第3章: React の基礎

この章では、React（リアクト）の基本的な概念をゼロから学びます。React はユーザーインターフェース（UI：ユーザーが見て操作する画面部分）を効率的に作るためのライブラリ（Library：特定の機能を提供するプログラムの集まり）です。

この章で学ぶこと

概念	一言で言うと	身近な例え
コンポーネント	画面の部品	レゴブロック。小さな部品を組み合わせてページを作る
JSX / TSX	HTMLのようにUIを書く構文	HTMLとJavaScriptを混ぜて書ける魔法の書き方
Props	親から子へのデータ受け渡し	上司から部下への指示書。「このデータを使って表示して」
State	コンポーネントの内部データ	変化する黒板。書き換えると画面が自動で更新される
イベント	ユーザー操作への反応	ボタンを押す→何かが起きる、という仕組み
useEffect	画面描画以外の処理	画面が表示された後に行う「裏方の仕事」（API通信など）
カスタムフック	共通ロジックの切り出し	よく使う手順をマニュアルにまとめておくこと

Reactを学ぶ前に： React は「宣言的UI」（Declarative UI）という考え方を採用しています。これは「画面がどうあるべきか」を記述するスタイルで、jQuery（ジェイクエリー）のように「画面のどこをどう操作するか」を記述する「命令的UI」（Imperative UI）とは大きく異なります。最初は戸惑うかもしれませんが、慣れると非常に直感的に感じるようになります。

目次

- 0. 前提知識: HTMLとDOMの超基礎
- 1. React とは
- 2. JSX / TSX
- 3. コンポーネント
- 4. State (useState)
- 5. イベントハンドリング
- 6. useEffect

- 7. カスタムフック
- 8. よくあるミスと対処法

0. 前提知識: HTMLとDOMの超基礎

React は「画面を作る」ための仕組みです。「画面」とは内部的には **HTML** で記述された文書です。React のコードを読む前に、HTML と DOM について最低限おさえておきましょう。

0.1 HTMLって何？

HTML (HyperText Markup Language) は、Webページの構造を記述する言語です。 **<タグ名>...</タグ名>** という形で、文章のどの部分が「見出し」で、どの部分が「段落」で、どこに「リンク」があるか、をコンピュータに教えます。

```
<!DOCTYPE html>
<html>
  <head>
    <title>はじめてのページ</title>
  </head>
  <body>
    <h1>こんにちは</h1>
    <p>これは段落 (paragraph) です。</p>
    <a href="https://example.com">リンク</a>
  </body>
</html>
```

▼ ブラウザで表示すると：

こんにちは	← <h1> の部分が大きな見出し
これは段落 (paragraph) です。	← <p> の部分は普通の段落
リンク	← <a> の部分は青い下線付きリンク

0.2 よく出てくるHTMLタグ

タグ	意味	サンプル
<code><div></code>	汎用ブロック（箱）	<code><div>...</div></code>
<code></code>	汎用インライン（行内）	<code>太字</code>
<code><h1></code> ~ <code><h6></code>	見出し（h1が一番大きい）	<code><h1>タイトル</h1></code>
<code><p></code>	段落	<code><p>本文</p></code>
<code><a></code>	リンク	<code>テキスト</code>
<code></code> <code></code>	順序なしリスト	<code>項目</code>
<code><button></code>	ボタン	<code><button>OK</button></code>
<code><input></code>	入力欄	<code><input type="text" /></code>
<code><form></code>	フォーム	<code><form>...</form></code>
<code></code>	画像	<code></code>

タグには **属性（attribute）** を付けられます。`<a href="https://example.com">` の `href="..."` の部分が属性です。

0.3 DOM（Document Object Model）

ブラウザは HTML を読み込むと、それを**ツリー構造のオブジェクト**として記憶します。これが **DOM** です。

```
html
├─ body
│   ├── h1 ("こんにちは")
│   ├── p ("これは段落です。")
│   └─ a ("リンク") [href="https://example.com"]
```

「DOM を操作する」とは、JavaScript からこのツリーを読み書きして画面を変更することです。React は「DOM 操作を直接書かなくて済むようにする」ためのライブラリ、と言い換えてもOKです。

0.4 ブラウザの開発者ツールで触ってみる

Chrome や Edge で右クリック →「検証（Inspect）」を選ぶと、開発者ツール（DevTools）が開きます。「Elements」タブが今そのページの DOM ツリーです。**Console** タブで JavaScript を実行できます。

```
// Consoleタブで実行
document.title           // ▶ ページタイトルを取得
document.querySelector("h1").textContent // ▶ h1の文字を取得
```

これだけ覚えれば次に進める: HTMLは「タグで文書の構造を書くもの」、DOMは「ブラウザがそれを読み込んだ後のツリー状のデータ」、React は「DOMを直接いじらず、JSX という書き方で画面を宣言するライブラリ」。これだけ頭に入れて先へ進みましょう。

1. React とは

1.1 コンポーネントベースの考え方

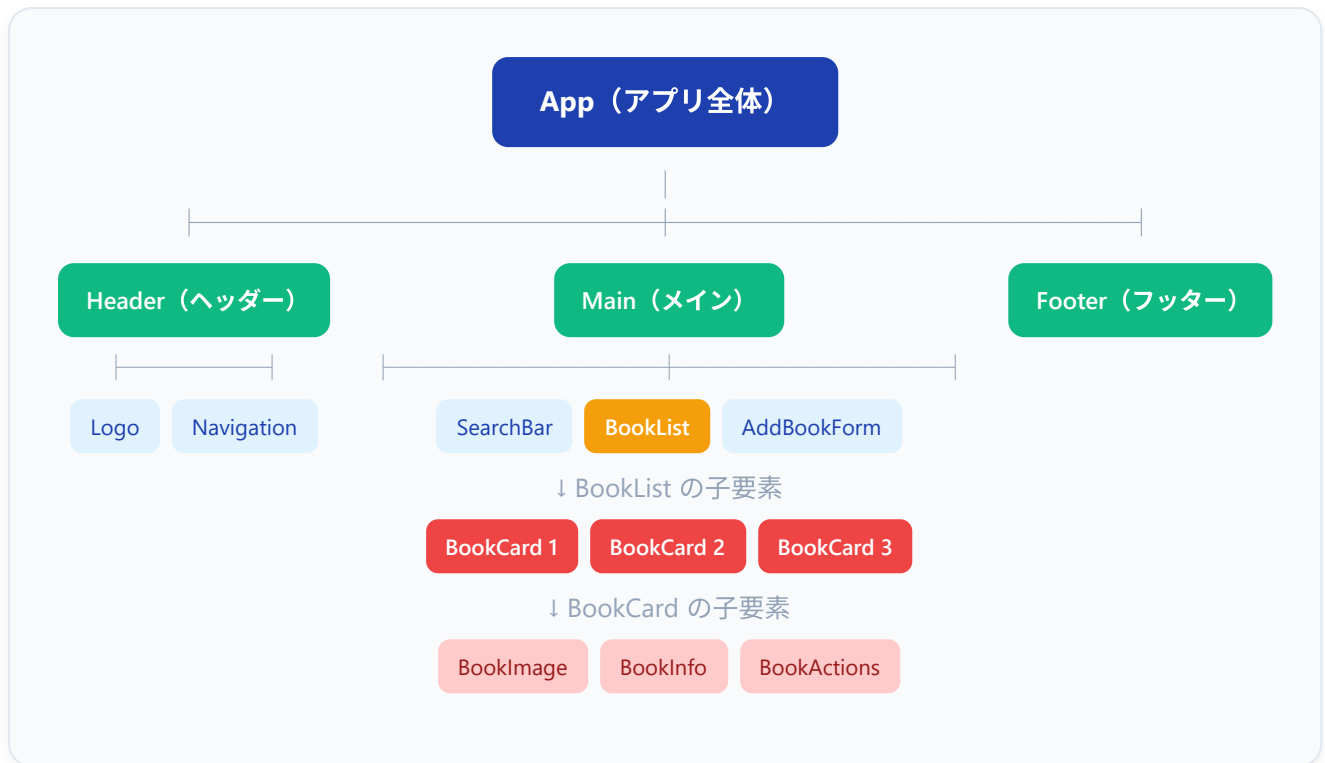
React は Facebook（現 Meta） が開発した、ユーザーインターフェース（UI：画面）を構築するための JavaScript ライブラリです。2013年にオープンソース（誰でも無料で使える形）として公開され、現在では世界で最も使われているUIライブラリです。

React の最大の特徴は「**コンポーネント**」（Component：UIの部品。ボタン、カード、ヘッダーなど、画面を構成する一つひとつの要素）という考え方です。

なぜReactが人気なの？ 従来のWeb開発では、HTMLファイル全体を書き換える必要がありました。React では、変化した部分だけを効率的に更新できるため、**高速で滑らかな画面更新**が可能です。また、一度作った部品（コンポーネント）を使い回せるため、開発効率も大幅に向上します。

コンポーネントとは、UI の一部分を独立した再利用可能なパーツとして定義したものです。レゴブロックのように、小さなパーツを組み合わせで画面全体を構築します。

例えば、書籍管理アプリを考えてみましょう。



このように、画面を「コンポーネントの木構造（ツリー）」として捉えます。それぞれのコンポーネントは：

- **独立している**: 自分の状態（state）とロジックを持つ
- **再利用できる**: **BookCard** は何度でも使い回せる
- **組み合わせできる**: 小さなコンポーネントを組み合わせで大きなコンポーネントを作る

1.2 仮想DOM の仕組み

ブラウザが持つ **DOM（Document Object Model）** を直接操作すると、非常に遅くなります。React は「仮想DOM（Virtual DOM）」という仕組みを使って、この問題を解決します。

従来の方法

データ変更



DOM全体を再描画



ブラウザがレイアウト再計算



画面全体がちらつく・遅い

React の方法

データ変更 (state更新)



新しい仮想DOMを作成



前回の仮想DOMと比較
(差分検出 = Reconciliation)



変更箇所だけ実際のDOMに反映



最小限の更新で高速に描画

もう少し詳しく見てみましょう。



ポイント: 仮想DOM は JavaScript のオブジェクトに過ぎません。JavaScript オブジェクトの比較は、実際の DOM 操作よりもはるかに高速です。React がこの差分を計算し、必要最小限の DOM 操作だけを行うことで、パフォーマンスが向上します。

1.3 なぜ React を選んだのか

フロントエンドのフレームワーク/ライブラリには多くの選択肢があります。

特徴	React	Vue	Angular	Svelte
学習コスト	中	低	高	低
エコシステム	非常に大きい	大きい	大きい	成長中
求人数	非常に多い	多い	多い	少なめ
TypeScript対応	良好	良好	標準搭載	良好
コミュニティ	最大級	大きい	大きい	成長中

React を選ぶ理由:

1. **巨大なエコシステム:** npm パッケージ、ツール、ライブラリが圧倒的に多い
2. **求人需要:** 日本でもグローバルでも、React エンジニアの需要は非常に高い

3. 学んだ知識の応用範囲が広い: React Native（モバイルアプリ）、Next.js（フルスタック）など
4. TypeScript との親和性: 型安全な開発が自然にできる
5. 豊富な学習リソース: 公式ドキュメント、書籍、動画、コミュニティが充実

2. JSX / TSX

2.1 JSX とは何か

JSX（JavaScript XML）は、JavaScript の中に HTML のようなコードを書ける構文拡張です。TypeScript で使う場合は TSX と呼びます（ファイル拡張子が `.tsx`）。

JSX は実際にはブラウザが直接理解できるものではなく、ビルドツール（Vite, Webpack など）によって通常の JavaScript に変換されます。

```
// JSX で書いたコード
const element = <h1>こんにちは、React！</h1>;

// ↓ ビルドツールによって変換される ↓

// 実際に行われる JavaScript コード
const element = React.createElement("h1", null, "こんにちは、React！");
```

JSX のおかげで、UI の構造を直感的に記述できます。

2.2 基本的な構文

最小の React コンポーネント

```
// =====
// 一番小さい React コンポーネントの例
// =====
// function = JS の関数宣言キーワード
// App      = 関数（コンポーネント）の名前。React では「大文字で始まる関数」が
//            自動的にコンポーネントとして扱われる。小文字始まりだと普通の関数。
// ()       = 引数の括弧（このコンポーネントは引数を受け取らないので空）
// { ... }  = 関数の本体ブロック
function App() {
  // return = この関数が返す値を指定するキーワード
  // <h1>...</h1> は JSX。HTML そっくりだが、内部では React.createElement に変換される。
  return <h1>Hello, World!</h1>;
}
```

▼ ブラウザでの表示:

画面には「Hello, World!」という大きな見出しが1行表示されます。

はHTMLで一番大きな見出しを意味するため、文字サイズが大きく、太字になります。

複数の要素を返す場合

JSX では、必ず1つのルート要素 で囲む必要があります。

```
// =====
// ❌ NG例 - 複数の要素を「並べただけ」では返せない
// =====
// JSX は内部的には1個のオブジェクトを返す JavaScript 式に変換される。
// 並列に複数の要素を書くと「どれが戻り値？」と JS が判断できずエラーになる。
function App() {
  return (
    <h1>タイトル</h1>      /* ← 1つ目のルート要素 */
    <p>本文</p>             /* ← 2つ目のルート要素 → ❌ エラー */
  );
}
// ▼ エラー
// JSX expressions must have one parent element.
```

```
// =====
// ✅ 解決策1: <div> で1つに包む
// =====
function App() {
  return (
    // (1) ルート要素は1つ。<div> でまとめる。
    // JSX のインデントは見やすさのため、必須ではない。
    <div>
      <h1>タイトル</h1>    /* (2) <div> の中の子要素1つ目 */
      <p>本文</p>           /* (3) <div> の中の子要素2つ目 */
    </div>
  );
}
```

▼ ブラウザでの表示:

タイトル ← <h1> 大きな見出し 本文 ← <p> 通常の段落 ただし HTML 出力に余計な <div> タグが1つ増える点に注意。

```
// =====
// ✅ 解決策2: Fragment (<>...</>) で包む【こちらが推奨】
// =====
// Fragment は「論理的に1つにまとめるが、DOMには何も出さない」 特殊なラッパー。
function App() {
  return (
    <>                                {/* 開始タグ: 名前のないタグ */}
      <h1>タイトル</h1>
      <p>本文</p>
    </>                                {/* 終了タグ */}
  );
}
```

▼ ブラウザでの表示:

上の `<div>` 版と見た目は同じだが、実際の HTML には `<div>` が出力されない。レイアウト崩れを起こしたくないときに便利。

HTML と JSX の違い

```
// =====
// HTML との細かい違い - JSX で書き換えが必要なポイント
// =====
function App() {
  return (
    <div>
      {/*
        (1) class は予約語のため className を使う
            → JS の class (クラス定義) と区別するため。
      */}
      <div className="container">

        {/*
          (2) <label for=""> は htmlFor に書き換える
              → for も JS の予約語 (for ループ) だから。
        */}
        <label htmlFor="name">名前:</label>
        <input id="name" />

        {/*
          (3) style 属性はオブジェクトで渡す (文字列ではない!)
          - 外側の {} は「JSXに JS の式を埋め込む」記号
          - 内側の {} は「JS のオブジェクトリテラル」
          - キーはキャメルケース: font-size → fontSize, margin-top → marginTop
          - 値は文字列 ("16px") でもOK。数値だけ渡すと px が自動で付くものもある
        */}
        <p style={{ color: "red", fontSize: "16px", marginTop: "10px" }}>
          赤いテキスト
        </p>

        {/*
          (4) コメントは {/* ... */} 形式
              普通の HTML コメント <!-- ... --> は JSX では書けない。
        */}

        {/*
          (5) 自己閉じタグは「/」で必ず閉じる必要がある
              HTML では <br>,  でも OK だが、JSX は厳密。
        */}
        <br />
        
        <input type="text" />
      </div>
    </div>
  );
}
```

▼ ブラウザでの表示:

`` 名前: [] ← input ボックス

赤いテキスト ← color:red, font-size:16px で表示

[ロゴ画像]

[] ← 別の input ボックス ``

HTML	JSX/TSX
<code>class</code>	<code>className</code>
<code>for</code>	<code>htmlFor</code>
<code>style="color: red"</code>	<code>style={{ color: "red" }}</code>
<code>tabindex</code>	<code>tabIndex</code>
<code>onclick</code>	<code>onClick</code>
<code><!-- コメント --></code>	<code>{/* コメント */}</code>

2.3 JavaScript 式の埋め込み

JSX の中で `{ }` を使うと、JavaScript の式を埋め込みます。

```
// =====
// JSX に JavaScript の式を埋め込むサンプル
// =====
// JSX 中で { ... } を使うと、その中だけ「JavaScriptの式」を書ける。
// 注意: 文 (statement) は書けない。式 (expression) のみ。
//   ・OK : 変数、計算式、関数呼び出し、三項演算子、配列メソッド
//   ・NG : if 文、for 文、変数宣言 (const x = ...)

function App() {
  // (1) コンポーネント関数の中 (return より前) は普通の JS。
  //     ここで変数宣言や計算をしておく。
  const userName: string = "田中太郎"; // 文字列の変数
  const age: number = 25; // 数値の変数
  const today: Date = new Date(); // 現在時刻の Date オブジェクト

  // (2) return 内の JSX に { } で値を差し込む
  return (
    <div>
      {/* (a) 変数をそのまま埋め込む。
        <h1> の中身は「ようこそ、田中太郎さん！」になる。 */}
      <h1>ようこそ、{userName}さん!</h1>

      {/* (b) 計算式の結果を埋め込む。
        {age + 1} は 25+1 を計算してから 26 を表示する。 */}
      <p>年齢: {age}歳 (来年は{age + 1}歳)</p>

      {/* (c) メソッド呼び出しも埋め込める。
        today.toLocaleDateString("ja-JP") は日付を「2026/3/16」形式に変換する。 */}
      <p>今日の日付: {today.toLocaleDateString("ja-JP")}</p>

      {/* (d) テンプレートリテラル (バッククォート) も式なのでOK。 */}
      <p>`${userName}さんは${age}歳です`</p>
    </div>
  );
}
```

▼ ブラウザでの表示（2026年5月10日に開いた場合の例）：

`` ようこそ、田中太郎さん！ ←

大きな見出し

年齢: 25歳（来年は26歳）

今日の日付: 2026/5/10

田中太郎さんは25歳です ``

重要: `{ }` の中に書けるのは 式（expression）だけです。 `if` 文や `for` 文などの 文（statement）は書けません。

```
// NG: if 文は式ではないので書けない
<p>{if (age >= 20) { "成人" }}</p>

// OK: 三項演算子は式なので書ける
<p>{age >= 20 ? "成人" : "未成年"}</p>
```

2.4 条件付きレンダリング

三項演算子

```
// =====  
// 三項演算子 ?: で表示を切り替える  
// =====  
// 条件 ? 真の時の値 : 偽の時の値 という JS の式。  
// JSX の中で「片方を表示・もう片方を表示」を切り替えるのに最適。  
  
function UserStatus() {  
  // (1) ユーザーがログイン済みかどうかを表す状態（ここでは固定値）  
  const isLoggedIn: boolean = true;  
  const userName: string = "田中太郎";  
  
  return (  
    <div>  
      {/*  
        (2) JSX 内の { ... } は JS の式。三項演算子は式なので使える。  
        isLoggedIn が true なら <p>ようこそ...</p> を、  
        false なら <p>ログインしてください。</p> を返す。  
      */}  
      {isLoggedIn ? (  
        <p>ようこそ、{userName}さん！</p>  
      ) : (  
        <p>ログインしてください。</p>  
      )}  
    </div>  
  );  
}
```

▼ ブラウザでの表示:

- `isLoggedIn = true` のとき: ようこそ、田中太郎さん！
- `isLoggedIn = false` のとき: ログインしてください。

&&（論理AND）演算子

条件を満たしたときだけ表示したい場合に便利です。

```
// =====
// && 演算子で「条件を満たしたときだけ」表示する
// =====
// JS では: A && B
//   A が truthy (真っぽい値) なら B が評価され、その結果が返る。
//   A が falsy (false, 0, "", null, undefined) なら A 自身が返る。
// JSX では false / null / undefined は「何も表示しない」扱いになる。
// よって「条件 && JSX」と書けば「条件が true のときだけ JSX を表示」になる。

function Notification() {
  // 数値の状態 (未読件数) と真偽値の状態 (エラー有無)
  const unreadCount: number = 5;
  const hasError: boolean = false;

  return (
    <div>
      <h1>ダッシュボード</h1>

      {/*
        (a) unreadCount > 0 の結果は boolean。
          - 5 > 0 → true なので <p>...</p> が表示される
          - 0 > 0 → false なので何も表示されない
      */}
      {unreadCount > 0 && (
        <p className="notification">
          未読メッセージが{unreadCount}件あります。
        </p>
      )}

      {/*
        (b) hasError は false なので右辺は評価されず、何も表示されない
      */}
      {hasError && (
        <p className="error">エラーが発生しました。</p>
      )}
    </div>
  );
}
```

▼ ブラウザでの表示:

ダッシュボード ← <h1> 未読メッセージが5件あります。 ← (a) は表示される ← (b) hasError=false
なので非表示

注意: && 演算子を使う際、左辺が数値の 0 だと 0 がそのまま画面に表示されてしまいます。

```
// NG: count が 0 のとき、画面に「0」が表示されてしまう
{count} && <p>{count}件</p>

// OK: 比較演算子を使えば安全
{count} > 0 && <p>{count}件</p>
```

複雑な条件分岐

```
type Status = "loading" | "success" | "error";

function DataDisplay() {
  const status: Status = "success";
  const data: string = "データの内容";
  const errorMessage: string = "";

  // 関数で条件分岐をまとめる
  const renderContent = (): JSX.Element => {
    switch (status) {
      case "loading":
        return <p>読み込み中...</p>;
      case "error":
        return <p className="error">エラー: {errorMessage}</p>;
      case "success":
        return <p>{data}</p>;
    }
  };

  return (
    <div>
      <h2>データ表示</h2>
      {renderContent()}
    </div>
  );
}
```

`status` が `"success"` の場合、画面には「データ表示」という見出しと「データの内容」というテキストが表示されます。

2.5 リストのレンダリング (map)

配列データを画面に表示するには、`map` メソッドを使います。

```
function FruitList() {  
  const fruits: string[] = ["りんご", "バナナ", "みかん", "ぶどう"];  
  
  return (  
    <div>  
      <h2>フルーツ一覧</h2>  
      <ul>  
        {fruits.map((fruit, index) => (  
          <li key={index}>{fruit}</li>  
        ))}  
      </ul>  
    </div>  
  );  
}
```

この結果、画面には「フルーツ一覧」という見出しと、箇条書きで以下のリストが表示されます:

- りんご
- バナナ
- みかん
- ぶどう

オブジェクト配列のレンダリング

```
// =====
// オブジェクト配列を map で展開して表示するサンプル
// =====

// (1) Book 型を定義（書籍データ1件分の形）
//     type を使うのが React/TS では一般的。interface でも同じことができる。
type Book = {
  id: number;           // 一意なID (key に使う)
  title: string;        // 書籍タイトル
  author: string;       // 著者名
  price: number;        // 税込価格
  isAvailable: boolean; // 在庫があるか (true=あり, false=なし)
};

function BookList() {
  // (2) Book 型の配列を作る。
  //     型注釈 Book[] で「Book型のオブジェクトしか入らない配列」と宣言。
  //     1件でもプロパティが欠けていると VS Code が赤線で教えてくれる。
  const books: Book[] = [
    { id: 1, title: "React入門",      author: "田中太郎", price: 2800, isAvailable: true },
    { id: 2, title: "TypeScript実践", author: "鈴木花子", price: 3200, isAvailable: false },
    { id: 3, title: "Next.js徹底解説", author: "佐藤一郎", price: 3500, isAvailable: true },
  ];

  return (
    <div>
      <h2>書籍一覧</h2>

      {/*
        (3) books.map((book) => JSX) で「配列の各要素を JSX に変換」する。
        戻り値は「JSX要素の配列」になり、それを React がそのまま並べて描画する。

        (4) key={book.id} は「リスト項目を識別するため」の特別な属性。
        React がリストの差分検出に使う。配列内で一意である必要がある。
        添え字 i ではなく、データのID (DBのprimary key) を使うのが鉄則。
      */}
      {books.map((book) => (
        <div key={book.id} className="book-card">
          <h3>{book.title}</h3>
          <p>著者: {book.author}</p>

          {/*
            (5) book.price.toLocaleString() は数値を「3桁ごとにカンマで区切った
            文字列」に変換するメソッド。2800 → "2,800"
          */}
          <p>価格: ¥{book.price.toLocaleString()}</p>

          {/*
```

(6) 三項演算子で表示文字列と色を切り替える

`{" "}` は意図的に空白を1文字入れたいとき使う書き方

```
*/}  
<p>  
  状態:{" "  
  <span style={{ color: book.isAvailable ? "green" : "red" }}>  
    {book.isAvailable ? "在庫あり" : "在庫なし"}  
  </span>  
</p>  
</div>  
  )}  
</div>  
);  
}
```

▼ ブラウザでの表示:

書籍一覧 ————— React入門 著者: 田中太郎 価格: ¥2,800 状態:
在庫あり ← 緑色 ————— TypeScript実践 著者: 鈴木花子 価格:
¥3,200 状態: 在庫なし ← 赤色 ————— Next.js徹底解説 著者: 佐
藤一郎 価格: ¥3,500 状態: 在庫あり ← 緑色

重要: `key` プロパティには、配列内で一意（ユニーク）な値を指定します。データベースから取得した `id` がベストです。
`index` は最後の手段です（要素の並び替え・追加・削除で不具合の原因になります）。

フィルタリングと map の組み合わせ

```
function AvailableBookList() {
  const books: Book[] = [
    { id: 1, title: "React入門", author: "田中太郎", price: 2800, isAvailable: true },
    { id: 2, title: "TypeScript実践", author: "鈴木花子", price: 3200, isAvailable: false },
    { id: 3, title: "Next.js徹底解説", author: "佐藤一郎", price: 3500, isAvailable: true },
  ];

  return (
    <div>
      <h2>在庫のある書籍</h2>
      {books
        .filter((book) => book.isAvailable)
        .map((book) => (
          <div key={book.id}>
            <p>
              {book.title} - ¥{book.price.toLocaleString()}
            </p>
          </div>
        ))}
    </div>
  );
}
```

この結果、画面には「在庫のある書籍」という見出しと、在庫のある2冊だけが表示されます:

React入門 - ¥2,800

Next.js徹底解説 - ¥3,500

3. コンポーネント

3.1 関数コンポーネントの書き方

React では、関数コンポーネント が標準的な書き方です（クラスコンポーネントは現在の React では推奨されていません）。

```
// 最もシンプルな関数コンポーネント
function Greeting() {
  return <h1>こんにちは! </h1>;
}

// アロー関数でも書ける
const Greeting = () => {
  return <h1>こんにちは! </h1>;
};

// 1行で返せる場合は return を省略できる
const Greeting = () => <h1>こんにちは! </h1>;
```

いずれの書き方でも、画面には「こんにちは！」という見出しが表示されます。

コンポーネントの使い方

```
function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
      <Greeting />
    </div>
  );
}
```

この結果、画面には「こんにちは！」が3回表示されます。同じコンポーネントを何度でも再利用できます。

命名規則: コンポーネント名は必ず **大文字始まり**（PascalCase）にします。小文字で始めると、HTML タグとして認識されてしまいます。

```
// OK: 大文字始まり → React コンポーネント
<Greeting />
<BookCard />
<UserProfile />

// NG: 小文字始まり → HTML タグとして扱われる
<greeting /> // <greeting> という存在しない HTML タグになる
```

3.2 Props の受け渡し（TypeScript での型定義含む）

Props（プロパティ）は、親コンポーネントから子コンポーネントにデータを渡す仕組みです。

基本的な Props

```
// =====
// Props の最小サンプル - 親が「name」という値を子に渡す
// =====

// (1) 子コンポーネントが受け取る Props の「形」を type で定義する。
//     ここでは「name という string が必須で1つ」と宣言。
type GreetingProps = {
  name: string;
};

// (2) Greeting 関数コンポーネント
//     関数の引数で「props オブジェクト」を受け取る。
//     ここでは（{ name }: GreetingProps）と分割代入 + 型注釈で書いている。
//     等価なのは:
//     function Greeting(props: GreetingProps) {
//       const name = props.name;
//       ...
//     }
//     props.name と書かずに済むので分割代入のほうが好まれる。
function Greeting({ name }: GreetingProps) {
  // (3) 受け取った name を JSX 内に { } で埋め込んで使う
  return <h1>こんにちは、{name}さん！</h1>;
}

// (4) 親コンポーネント (App) が Greeting を3回使う
//     <Greeting name="田中" /> の name="田中" の部分が Props として子に渡る。
//     文字列リテラルは "... " または '...' で書く（{} はJS式埋め込み用）。
function App() {
  return (
    <div>
      <Greeting name="田中" />
      <Greeting name="鈴木" />
      <Greeting name="佐藤" />
    </div>
  );
}
```

▼ ブラウザでの表示:

こんにちは、田中さん！ ← <h1> こんにちは、鈴木さん！ ← <h1> こんにちは、佐藤さん！ ← <h1>

同じ Greeting コンポーネントが3回呼ばれ、それぞれ異なる name を受け取って描画されている。コンポーネントの再利用性がこれで実現できる。

さまざまな型の Props

```
type UserCardProps = {
  name: string;           // 必須の文字列
  age: number;            // 必須の数値
  email?: string;         // オプション (省略可能)
  isAdmin: boolean;      // 必須の真偽値
  hobbies: string[];      // 文字列の配列
  onClickProfile: () => void; // 関数 (コールバック)
};

function UserCard({
  name,
  age,
  email,
  isAdmin,
  hobbies,
  onClickProfile,
}: UserCardProps) {
  return (
    <div className="user-card">
      <h2>{name}</h2>
      <p>年齢: {age}歳</p>

      {/* オプションな props は存在チェック */}
      {email && <p>メール: {email}</p>}

      <p>権限: {isAdmin ? "管理者" : "一般ユーザー"}</p>

      <div>
        趣味:
        <ul>
          {hobbies.map((hobby, index) => (
            <li key={index}>{hobby}</li>
          ))}
        </ul>
      </div>

      <button onClick={onClickProfile}>プロフィールを見る</button>
    </div>
  );
}

// 使い方
function App() {
  const handleClick = () => {
    alert("プロフィールページへ移動します");
  };

  return (
```

```
<UserCard
  name="田中太郎"
  age={25}
  email="tanaka@example.com"
  isAdmin={false}
  hobbies={["読書", "プログラミング", "映画鑑賞"]}
  onClickProfile={handleClick}
/>
);
}
```

この結果、画面には以下のように表示されます:

田中太郎

年齢: 25歳

メール: tanaka@example.com

権限: 一般ユーザー

趣味: - 読書 - プログラミング - 映画鑑賞

【プロフィールを見る】ボタン

デフォルト値を持つ Props

```
type ButtonProps = {
  label: string;
  color?: string;
  size?: "small" | "medium" | "large";
  disabled?: boolean;
};

function Button({
  label,
  color = "blue",
  size = "medium",
  disabled = false,
}: ButtonProps) {
  const sizeStyles: Record<string, string> = {
    small: "8px 16px",
    medium: "12px 24px",
    large: "16px 32px",
  };

  return (
    <button
      style={{
        backgroundColor: color,
        padding: sizeStyles[size],
        color: "white",
        border: "none",
        borderRadius: "4px",
        opacity: disabled ? 0.5 : 1,
        cursor: disabled ? "not-allowed" : "pointer",
      }}
      disabled={disabled}
    >
      {label}
    </button>
  );
}

function App() {
  return (
    <div>
      <Button label="デフォルト" />
      <Button label="赤い大きなボタン" color="red" size="large" />
      <Button label="無効化" disabled />
      <Button label="緑の小さなボタン" color="green" size="small" />
    </div>
  );
}
```

この結果、画面には4つのボタンが表示されます:

1. 「デフォルト」 - 青色・中サイズのボタン
2. 「赤い大きなボタン」 - 赤色・大サイズのボタン
3. 「無効化」 - 青色・中サイズだが半透明でクリック不可のボタン
4. 「緑の小さなボタン」 - 緑色・小サイズのボタン

children Props

コンポーネントのタグで囲んだ中身は、`children` として渡されます。

```

type CardProps = {
  title: string;
  children: React.ReactNode;
};

function Card({ title, children }: CardProps) {
  return (
    <div
      style={{
        border: "1px solid #ddd",
        borderRadius: "8px",
        padding: "16px",
        margin: "8px",
      }}
    >
      <h2 style={{ borderBottom: "1px solid #eee", paddingBottom: "8px" }}>
        {title}
      </h2>
      <div>{children}</div>
    </div>
  );
}

function App() {
  return (
    <div>
      <Card title="お知らせ">
        <p>新機能がリリースされました！</p>
        <p>詳しくはこちらをご覧ください。</p>
      </Card>

      <Card title="プロフィール">
        
        <p>田中太郎</p>
      </Card>
    </div>
  );
}

```

この結果、画面には2つのカードが表示されます:

【お知らせカード】 枠線で囲まれた領域に「お知らせ」という見出しと、2行のテキスト

【プロフィールカード】 枠線で囲まれた領域に「プロフィール」という見出しと、画像・名前

3.3 コンポーネントの分割の考え方

コンポーネントを分割する判断基準:

1. 再利用されるか: 複数箇所で使う UI は別コンポーネントにする
2. 責務が明確か: 1つのコンポーネントが1つの役割を持つ
3. 複雑すぎないか: 1ファイルが 100行を超えたら分割を検討する
4. 独立してテストできるか: テストしやすい単位に分ける

```
src/
├── components/
│   ├── common/           # 汎用コンポーネント
│   │   ├── Button.tsx
│   │   ├── Card.tsx
│   │   ├── Input.tsx
│   │   └── Modal.tsx
│   ├── layout/          # レイアウト系
│   │   ├── Header.tsx
│   │   ├── Footer.tsx
│   │   └── Sidebar.tsx
│   └── book/             # 書籍機能関連
│       ├── BookCard.tsx
│       ├── BookList.tsx
│       ├── BookDetail.tsx
│       └── BookForm.tsx
├── pages/                # ページ単位
│   ├── HomePage.tsx
│   ├── BookListPage.tsx
│   └── BookDetailPage.tsx
└── App.tsx
```

3.4 書籍カードコンポーネントの例

実際のアプリで使うような、少し本格的なコンポーネントを作ってみましょう。

```
// types.ts - 型定義
type Book = {
  id: number;
  title: string;
  author: string;
  price: number;
  rating: number;          // 1~5
  isAvailable: boolean;
  coverImage?: string;
  publishedDate: string;
  tags: string[];
};

// BookCard.tsx - 書籍カードコンポーネント
type BookCardProps = {
  book: Book;
  onAddToCart: (bookId: number) => void;
  onToggleFavorite: (bookId: number) => void;
  isFavorite: boolean;
};

function BookCard({ book, onAddToCart, onToggleFavorite, isFavorite }: BookCardProps) {
  // 星の表示を作る関数
  const renderStars = (rating: number): string => {
    return "★".repeat(rating) + "☆".repeat(5 - rating);
  };

  return (
    <div
      style={{
        border: "1px solid #ddd",
        borderRadius: "12px",
        padding: "16px",
        maxWidth: "300px",
        boxShadow: "0 2px 8px rgba(0,0,0,0.1)",
      }}
    >
      { /* 表紙画像 */ }
      {book.coverImage ? (
        <img
          src={book.coverImage}
          alt={` ${book.title} の表紙`}
          style={{ width: "100%", borderRadius: "8px" }}
        />
      ) : (
        <div
          style={{
            width: "100%",
            height: "200px",

```

```

        backgroundColor: "#f0f0f0",
        borderRadius: "8px",
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        color: "#999",
    }}
    >
    No Image
</div>
)}

{/* 書籍情報 */}
<h3 style={{ margin: "12px 0 4px" }}>{book.title}</h3>
<p style={{ color: "#666", margin: "0 0 8px" }}>{book.author}</p>

{/* 評価 */}
<p style={{ color: "#f39c12", margin: "0 0 8px" }}>
    {renderStars(book.rating)} ({book.rating}/5)
</p>

{/* タグ */}
<div style={{ display: "flex", gap: "4px", flexWrap: "wrap", marginBottom: "8px" }}>
    {book.tags.map((tag) => (
        <span
            key={tag}
            style={{
                backgroundColor: "#e8f4fd",
                color: "#1a73e8",
                padding: "2px 8px",
                borderRadius: "12px",
                fontSize: "12px",
            }}
        >
            {tag}
        </span>
    ))}
</div>

{/* 価格と在庫状態 */}
<div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
    <span style={{ fontSize: "20px", fontWeight: "bold" }}>
        ¥{book.price.toLocaleString()}
    </span>
    <span style={{ color: book.isAvailable ? "green" : "red", fontSize: "14px" }}>
        {book.isAvailable ? "在庫あり" : "在庫なし"}
    </span>
</div>

{/* アクションボタン */}

```

```

<div style={{ display: "flex", gap: "8px", marginTop: "12px" }}>
  <button
    onClick={() => onAddToCart(book.id)}
    disabled={!book.isAvailable}
    style={{
      flex: 1,
      padding: "8px",
      backgroundColor: book.isAvailable ? "#1a73e8" : "#ccc",
      color: "white",
      border: "none",
      borderRadius: "6px",
      cursor: book.isAvailable ? "pointer" : "not-allowed",
    }}
  >
    カートに追加
  </button>
  <button
    onClick={() => onToggleFavorite(book.id)}
    style={{
      padding: "8px 12px",
      backgroundColor: "transparent",
      border: "1px solid #ddd",
      borderRadius: "6px",
      cursor: "pointer",
      fontSize: "18px",
    }}
  >
    {isFavorite ? "♥" : "♡"}
  </button>
</div>
</div>
);
}

```

// 使い方

```

function App() {
  const sampleBook: Book = {
    id: 1,
    title: "React入門ガイド",
    author: "田中太郎",
    price: 2800,
    rating: 4,
    isAvailable: true,
    publishedDate: "2025-01-15",
    tags: ["React", "TypeScript", "フロントエンド"],
  };

  return (
    <BookCard
      book={sampleBook}
    />
  );
}

```

```

    onAddToCart={({id}) => alert(`書籍ID: ${id} をカートに追加しました`)}
    onToggleFavorite={({id}) => alert(`書籍ID: ${id} のお気に入りを切り替えました`)}
    isFavorite={false}
  />
);
}

```

この結果、画面にはカード型のUIが表示されます:

【No Image】（灰色のプレースホルダー）

React入門ガイド

田中太郎

★★★★☆ (4/5)

React TypeScript フロントエンド （タグバッジ）

¥2,800 在庫あり（緑色）

【カートに追加】ボタン【♡】ボタン

4. State (useState)

4.1 state とは何か

state（状態）は、コンポーネントが持つ「変化するデータ」です。state が変わると、React は自動的にそのコンポーネントを再レンダリング（再描画）します。

普通の変数と state の違いを見てみましょう。

```

// NG: 普通の変数は変更しても再レンダリングされない
function Counter() {
  let count = 0;

  const handleClick = () => {
    count += 1; // 値は変わるが、画面は更新されない!
    console.log(count); // コンソールには 1, 2, 3... と出る
  };

  return (
    <div>
      <p>カウント: {count}</p> {/* ずっと 0 のまま */}
      <button onClick={handleClick}>+1</button>
    </div>
  );
}

```

この結果、ボタンをクリックしても画面には「**カウント: 0**」のまま変わりません。コンソールには値が増えていきますが、React は変数の変化を検知できないのです。

4.2 useState の使い方

```
// =====
// シンプルなカウンターコンポーネント（詳細コメント版）
// =====
// 「ボタンを押すと数値が +1 / -1 され、画面が自動更新される」
// React の最も基本的なインタラクティブ部品。
// =====

// React の useState フックを使うので import で取り込む。
// `useState` は「コンポーネントが値を覚えておく」ための関数。
import { useState } from "react";

// 関数コンポーネント = 大文字始まりの関数で、JSXを返すもの。
function Counter() {
  // -----
  // (1) state (状態) を作る
  // -----
  // useState<number>(0) は「数値型の状態を初期値0で作って」という意味。
  //
  // 戻り値は配列で2つの要素が入っている:
  //   [現在の値, 値を更新するための関数]
  //
  // この配列を「分割代入」で count, setCount に取り出している。
  //   → const count = 戻り値[0]; ← 現在の数値
  //   → const setCount = 戻り値[1]; ← 数値を変える関数
  //
  // 重要: count の値を直接変えることはできない (const なので再代入不可)。
  //       必ず setCount(...) を呼ぶ。setCount を呼ぶと、Reactが
  //       「state が変わったぞ」と認識し、自動で画面を再描画してくれる。
  const [count, setCount] = useState<number>(0);

  // -----
  // (2) ボタンが押されたときの処理（イベントハンドラ）
  // -----
  // アロー関数で「+1する処理」を定義。
  // setCount を呼ぶことで、count が 0 → 1 → 2 ... と更新される。
  const handleIncrement = () => {
    setCount(count + 1); // 「今の count + 1」を新しい値として設定
  };

  // 同じく「-1する処理」
  const handleDecrement = () => {
    setCount(count - 1);
  };

  // 「0 に戻す処理」
  const handleReset = () => {
    setCount(0);
  };
}
```

```
// -----
// (3) 画面に出すJSXを返す
// -----
// {count} の波カッコはJSX中にJavaScriptの値を埋め込む書き方。
// count が 5 なら、「カウント: 5」と表示される。
//
// onClick={handleDecrement} は「-1ボタンが押されたら handleDecrement を呼んで」の意味。
// 注意: onClick={handleDecrement()} と () を付けると「即実行」になりバグの元。
//      関数自体を渡す (= 関数名のみ書く) のが正解。
return (
  <div>
    <h2>カウンター</h2>
    <p>カウント: {count}</p>
    <button onClick={handleDecrement}>-1</button>
    <button onClick={handleReset}>リセット</button>
    <button onClick={handleIncrement}>+1</button>
  </div>
);
}
```

▼ ブラウザでの見た目（初期表示）：

カウンター	← <h2>
カウント: 0	← {count} に 0 が入る
[-1] [リセット] [+1]	← 3つの <button>

▼ 動作の流れ:

操作	count の変化	画面表示
初期表示	0	カウント: 0
「+1」を押す	0 → 1	カウント: 1
「+1」をもう1回	1 → 2	カウント: 2
「-1」を押す	2 → 1	カウント: 1
「リセット」を押す	1 → 0	カウント: 0

▼ 重要ポイント: 1. `count` は読み取り専用。直接 `count = 5;` とは書けない（書いても画面に反映されない） 2. 値を変えたいときは必ず `setCount(...)` を呼ぶ 3. `setCount` を呼ぶと、React が裏で関数 `Counter` をもう一度実行し、新しい count で画面を再描画する

useState の型推論

```
// 型を明示的に指定
const [count, setCount] = useState<number>(0);
const [name, setName] = useState<string>("");
const [isVisible, setIsVisible] = useState<boolean>(false);

// 初期値から型推論される (明示しなくてもOK)
const [count, setCount] = useState(0);           // number と推論
const [name, setName] = useState("");           // string と推論
const [isVisible, setIsVisible] = useState(false); // boolean と推論

// null を使う場合は明示的な型指定が必要
const [user, setUser] = useState<User | null>(null);
```

4.3 state の更新とリレンダリング



重要なポイント: `setCount` を呼んでも、その場では `count` の値は変わりません。次の再レンダリング時に新しい値になります。

```
function Counter() {
  const [count, setCount] = useState<number>(0);

  const handleClick = () => {
    setCount(count + 1);
    console.log(count); // まだ 0 のまま! (次のレンダリングで 1 になる)
    setCount(count + 1); // count はまだ 0 なので、0 + 1 = 1 になる (2 にはならない!)
  };

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={handleClick}>+2 のつもり (実際は+1) </button>
    </div>
  );
}
```

この結果、ボタンをクリックすると「カウント: 1」になります。+2 になると期待しますが、同じレンダリング内では `count` は古い値 (0) のままなので、両方とも `setCount(0 + 1) = setCount(1)` になります。

解決策: 関数型更新を使う

```
function Counter() {
  const [count, setCount] = useState<number>(0);

  const handleClick = () => {
    // prev には常に「最新の値」が渡される
    setCount((prev) => prev + 1); // 0 → 1
    setCount((prev) => prev + 1); // 1 → 2
  };

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={handleClick}>+2</button>
    </div>
  );
}
```

この結果、ボタンをクリックすると「カウント: 2」→「カウント: 4」→ ... と2ずつ増えます。関数型更新なら、直前の最新値に基づいて計算されるため、正しく動作します。

4.4 オブジェクトの state

```
import { useState } from "react";

type UserProfile = {
  name: string;
  email: string;
  age: number;
  bio: string;
};

function ProfileEditor() {
  const [profile, setProfile] = useState<UserProfile>({
    name: "田中太郎",
    email: "tanaka@example.com",
    age: 25,
    bio: "React を勉強中です",
  });

  // 名前を変更する関数
  const handleNameChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    // スプレッド構文で既存のプロパティをコピーし、name だけ上書き
    setProfile({
      ...profile,
      name: e.target.value,
    });
  };

  // メールを変更する関数
  const handleEmailChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setProfile({
      ...profile,
      email: e.target.value,
    });
  };

  // 年齢を変更する関数
  const handleAgeChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setProfile({
      ...profile,
      age: Number(e.target.value),
    });
  };

  // 自己紹介を変更する関数
  const handleBioChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
    setProfile({
      ...profile,
      bio: e.target.value,
    });
  };
}
```

```

};

return (
  <div>
    <h2>プロフィール編集</h2>

    <div>
      <label>名前: </label>
      <input value={profile.name} onChange={handleNameChange} />
    </div>

    <div>
      <label>メール: </label>
      <input value={profile.email} onChange={handleEmailChange} />
    </div>

    <div>
      <label>年齢: </label>
      <input
        type="number"
        value={profile.age}
        onChange={handleAgeChange}
      />
    </div>

    <div>
      <label>自己紹介: </label>
      <textarea value={profile.bio} onChange={handleBioChange} />
    </div>

    <h3>プレビュー</h3>
    <p>名前: {profile.name}</p>
    <p>メール: {profile.email}</p>
    <p>年齢: {profile.age}歳</p>
    <p>自己紹介: {profile.bio}</p>
  </div>
);
}

```

この結果、画面には4つの入力欄と、その下にプレビューが表示されます。入力欄に文字を入力すると、**リアルタイムにプレビュー部分が更新**されます。例えば名前の入力欄を「鈴木花子」に変更すると、プレビューの名前も即座に「名前: 鈴木花子」に変わります。

重要: オブジェクトの state を更新するときは、必ず**新しいオブジェクトを作成**してください。プロパティを直接変更してはいけません。

```
// NG: 直接変更 (React が変化を検知できない)
profile.name = "新しい名前";
setProfile(profile); // 同じオブジェクト参照なので再レンダリングされない!

// OK: 新しいオブジェクトを作成
setProfile({ ...profile, name: "新しい名前" });
```

ネストしたオブジェクトの更新

```
type Address = {
  prefecture: string;
  city: string;
  street: string;
};

type UserWithAddress = {
  name: string;
  address: Address;
};

function AddressEditor() {
  const [user, setUser] = useState<UserWithAddress>({
    name: "田中太郎",
    address: {
      prefecture: "東京都",
      city: "渋谷区",
      street: "道玄坂1-1-1",
    },
  });

  const handleCityChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    // ネストしたオブジェクトもスプレッド構文で展開する
    setUser({
      ...user,
      address: {
        ...user.address,
        city: e.target.value,
      },
    });
  };

  return (
    <div>
      <p>
        住所: {user.address.prefecture}
        {user.address.city}
        {user.address.street}
      </p>
      <label>市区町村: </label>
      <input value={user.address.city} onChange={handleCityChange} />
    </div>
  );
}
```

この結果、画面には「住所: 東京都渋谷区道玄坂1-1-1」と表示され、入力欄で市区町村を変更すると住所表示がリアルタイムに更新されます。例えば「新宿区」と入力すると、「住所: 東京都新宿区道玄坂1-1-1」に変わります。

4.5 配列の state

配列の state も、直接変更せず、常に**新しい配列を作成**して更新します。

```

import { useState } from "react";

type Todo = {
  id: number;
  text: string;
  completed: boolean;
};

function TodoApp() {
  const [todos, setTodos] = useState<Todo[]>([
    { id: 1, text: "React を学ぶ", completed: false },
    { id: 2, text: "TypeScript を学ぶ", completed: true },
  ]);
  const [inputValue, setInputValue] = useState<string>("");
  const [nextId, setNextId] = useState<number>(3);

  // — 追加 —
  const handleAdd = () => {
    if (inputValue.trim() === "") return;

    const newTodo: Todo = {
      id: nextId,
      text: inputValue,
      completed: false,
    };
    setTodos([...todos, newTodo]); // 既存の配列を展開して新しい要素を追加
    setInputValue("");
    setNextId(nextId + 1);
  };

  // — 削除 —
  const handleDelete = (id: number) => {
    setTodos(todos.filter((todo) => todo.id !== id)); // id が一致しない要素だけ残す
  };

  // — 完了/未完了の切り替え —
  const handleToggle = (id: number) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      )
    );
  };

  // — テキスト更新 —
  const handleUpdate = (id: number, newText: string) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id ? { ...todo, text: newText } : todo
      )
    );
  };
}

```

```

    )
  );
};

return (
  <div>
    <h2>TODOリスト</h2>

    {/* 入力欄 */}
    <div>
      <input
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
        placeholder="新しいタスクを入力"
      />
      <button onClick={handleAdd}>追加</button>
    </div>

    {/* TODOリスト */}
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => handleToggle(todo.id)}
          />
          <span
            style={{
              textDecoration: todo.completed ? "line-through" : "none",
              color: todo.completed ? "#999" : "#000",
            }}
          >
            {todo.text}
          </span>
          <button onClick={() => handleDelete(todo.id)}>削除</button>
        </li>
      ))}
    </ul>

    {/* 統計 */}
    <p>
      合計: {todos.length}件 / 完了: {todos.filter((t) => t.completed).length}件
      / 未完了: {todos.filter((t) => !t.completed).length}件
    </p>
  </div>
);
}

```

この結果、画面には以下のように表示されます:

TODOリスト

【入力欄】【追加ボタン】

- ☐ React を学ぶ【削除】
- ☒ ~~TypeScript を学ぶ~~【削除】

合計: 2件 / 完了: 1件 / 未完了: 1件

「テストを書く」と入力して【追加】を押すと:

- ☐ React を学ぶ【削除】
- ☒ ~~TypeScript を学ぶ~~【削除】
- ☐ テストを書く【削除】

合計: 3件 / 完了: 1件 / 未完了: 2件

「React を学ぶ」のチェックボックスをクリックすると:

- ☒ ~~React を学ぶ~~【削除】

合計: 3件 / 完了: 2件 / 未完了: 1件

配列操作の早見表

操作	使わない（直接変更）	使う（新しい配列を作成）
追加	<code>push</code> , <code>unshift</code>	<code>[...arr, newItem]</code> , <code>[newItem, ...arr]</code>
削除	<code>splice</code>	<code>filter</code>
置換	<code>arr[i] = value</code>	<code>map</code>
並び替え	<code>sort</code> , <code>reverse</code>	<code>[...arr].sort()</code> , <code>[...arr].reverse()</code>

5. イベントハンドリング

5.1 onClick

```
function ClickExamples() {
  // 基本的なクリックハンドラ
  const handleClick = () => {
    alert("ボタンがクリックされました!");
  };

  // 引数を受け取るクリックハンドラ
  const handleItemClick = (itemName: string) => {
    alert(`${itemName}がクリックされました`);
  };

  // イベントオブジェクトを受け取るクリックハンドラ
  const handleButtonClick = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log("クリック位置:", e.clientX, e.clientY);
    console.log("クリックされた要素:", e.currentTarget.textContent);
  };

  return (
    <div>
      {/* 基本的な使い方 */}
      <button onClick={handleClick}>クリック</button>

      {/* 引数を渡す場合はアロー関数で包む */}
      <button onClick={() => handleItemClick("Apple")}>Apple</button>
      <button onClick={() => handleItemClick("Banana")}>Banana</button>

      {/* イベントオブジェクトを使う */}
      <button onClick={handleButtonClick}>位置を表示</button>

      {/* インラインで直接書く */}
      <button onClick={() => console.log("インラインハンドラ")}>
        インライン
      </button>
    </div>
  );
}
```

この結果、画面には4つのボタンが表示されます。

【クリック】を押すと、アラートで「ボタンがクリックされました！」と表示されます。

【Apple】を押すと、「Appleがクリックされました」と表示されます。

【位置を表示】を押すと、コンソールにクリック座標が出力されます。

注意: `onClick` に関数を渡すときは、関数を実行してはいけません。

```
// NG: 関数を実行してしまっている (レンダリング時に即座に実行される)  
<button onClick={handleClick()}>クリック</button>
```

```
// OK: 関数の参照を渡す  
<button onClick={handleClick}>クリック</button>
```

```
// OK: アロー関数で包む  
<button onClick={() => handleClick()}>クリック</button>
```

5.2 onChange

```
import { useState } from "react";

function InputExamples() {
  const [text, setText] = useState<string>("");
  const [selectedColor, setSelectedColor] = useState<string>("red");
  const [isChecked, setIsChecked] = useState<boolean>(false);
  const [selectedSize, setSelectedSize] = useState<string>("M");

  return (
    <div>
      { /* テキスト入力 */ }
      <div>
        <label>名前: </label>
        <input
          type="text"
          value={text}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            setText(e.target.value)}
        />
        placeholder="名前を入力してください"
      </div>
      <p>入力値: 「{text}」 </p>
    </div>

    { /* セレクトボックス */ }
    <div>
      <label>色: </label>
      <select
        value={selectedColor}
        onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>
          setSelectedColor(e.target.value)}
      >
        <option value="red">赤</option>
        <option value="blue">青</option>
        <option value="green">緑</option>
      </select>
      <p style={{ color: selectedColor }}>
        選択した色: {selectedColor}
      </p>
    </div>

    { /* チェックボックス */ }
    <div>
      <label>
        <input
          type="checkbox"
          checked={isChecked}
        />
      </label>
    </div>
  );
}
```

```

        onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            setIsChecked(e.target.checked)
        }
    />
    利用規約に同意する
</label>
<p>同意状態: {isChecked ? "同意済み" : "未同意"}</p>
</div>

{ /* ラジオボタン */ }
<div>
    <p>サイズ:</p>
    {[ "S", "M", "L", "XL" ].map((size) => (
        <label key={size} style={{ marginRight: "12px" }}>
            <input
                type="radio"
                name="size"
                value={size}
                checked={selectedSize === size}
                onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
                    setSelectedSize(e.target.value)
                }
            />
            {size}
        </label>
    ))}
    <p>選択サイズ: {selectedSize}</p>
</div>
</div>
);
}

```

この結果、画面には4つの入力セクションが表示されます:

1. テキスト入力欄に「こんにちは」と入力すると、下に「入力値: 「こんにちは」」とリアルタイム表示
2. セレクトボックスで「青」を選ぶと、「選択した色: blue」が青色で表示
3. チェックボックスをオンにすると、「同意状態: 同意済み」に変化
4. ラジオボタンで「L」を選ぶと、「選択サイズ: L」に変化

5.3 onSubmit

```
import { useState } from "react";

type BookFormData = {
  title: string;
  author: string;
  price: string;
  category: string;
  description: string;
};

function BookForm() {
  const [formData, setFormData] = useState<BookFormData>({
    title: "",
    author: "",
    price: "",
    category: "programming",
    description: "",
  });

  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>({});
  const [isSubmitted, setIsSubmitted] = useState<boolean>(false);

  // バリデーション関数
  const validate = (): boolean => {
    const newErrors: Partial<Record<keyof BookFormData, string>> = {};

    if (formData.title.trim() === "") {
      newErrors.title = "タイトルは必須です";
    }
    if (formData.author.trim() === "") {
      newErrors.author = "著者は必須です";
    }
    if (formData.price === "" || Number(formData.price) <= 0) {
      newErrors.price = "価格は0より大きい数値を入力してください";
    }

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  // 汎用的な入力ハンドラ
  const handleChange = (
    e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement>
  ) => {
    const { name, value } = e.target;
    setFormData({
      ...formData,
      [name]: value,
    });
  };
}
```

```

});

// 入力時にエラーをクリア
if (errors[name as keyof BookFormData]) {
  setErrors({
    ...errors,
    [name]: undefined,
  });
}
};

// フォーム送信
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault(); // ページ遷移（デフォルト動作）を防ぐ

  if (validate()) {
    console.log("送信データ:", formData);
    setIsSubmitted(true);
    // 実際のアプリではここで API を呼ぶ
  }
};

if (isSubmitted) {
  return (
    <div>
      <h2>登録完了! </h2>
      <p>「{formData.title}」を登録しました。</p>
      <button onClick={() => setIsSubmitted(false)}>もう1冊登録する</button>
    </div>
  );
}

return (
  <form onSubmit={handleSubmit}>
    <h2>書籍登録</h2>

    <div>
      <label htmlFor="title">タイトル *</label>
      <input
        id="title"
        name="title"
        type="text"
        value={formData.title}
        onChange={handleChange}
      />
      {errors.title && <p style={{ color: "red" }}>{errors.title}</p>}
    </div>

    <div>
      <label htmlFor="author">著者 *</label>

```

```

    <input
      id="author"
      name="author"
      type="text"
      value={formData.author}
      onChange={handleChange}
    />
    {errors.author} && <p style={{ color: "red" }}>{errors.author}</p>
  </div>

  <div>
    <label htmlFor="price">価格 *</label>
    <input
      id="price"
      name="price"
      type="number"
      value={formData.price}
      onChange={handleChange}
    />
    {errors.price} && <p style={{ color: "red" }}>{errors.price}</p>
  </div>

  <div>
    <label htmlFor="category">カテゴリ</label>
    <select
      id="category"
      name="category"
      value={formData.category}
      onChange={handleChange}
    >
      <option value="programming">プログラミング</option>
      <option value="design">デザイン</option>
      <option value="business">ビジネス</option>
      <option value="other">その他</option>
    </select>
  </div>

  <div>
    <label htmlFor="description">説明</label>
    <textarea
      id="description"
      name="description"
      value={formData.description}
      onChange={handleChange}
      rows={4}
    />
  </div>

  <button type="submit">登録する</button>
</form>

```

```
);  
}
```

この結果、画面には書籍登録フォームが表示されます:

書籍登録

タイトル:【入力欄】 著者:【入力欄】 価格*:【入力欄】 カテゴリ:【セレクトボックス（プログラミング）】 説明:【テキストエリア】

【登録する】ボタン

何も入力せずに【登録する】を押すと、各必須項目の下に赤字で「タイトルは必須です」等のエラーメッセージが表示されます。

すべての必須項目を入力して送信すると、「登録完了！」画面に切り替わり、「React入門」を登録しました。」と表示されます。

5.4 イベントオブジェクトの型 一覧

イベント	型
<code>onClick</code> (ボタン)	<code>React.MouseEvent<HTMLButtonElement></code>
<code>onClick</code> (div)	<code>React.MouseEvent<HTMLDivElement></code>
<code>onChange</code> (input)	<code>React.ChangeEvent<HTMLInputElement></code>
<code>onChange</code> (select)	<code>React.ChangeEvent<HTMLSelectElement></code>
<code>onChange</code> (textarea)	<code>React.ChangeEvent<HTMLTextAreaElement></code>
<code>onSubmit</code> (form)	<code>React.FormEvent<HTMLFormElement></code>
<code>onKeyDown</code>	<code>React.KeyboardEvent<HTMLInputElement></code>
<code>onFocus</code> / <code>onBlur</code>	<code>React.FocusEvent<HTMLInputElement></code>

6. useEffect

6.1 副作用とは

React コンポーネントの主な仕事は「UI を描画すること（レンダリング）」です。それ以外の処理を副作用（Side Effect）と呼びます。

例: - API からデータを取得する - DOM を直接操作する - タイマーを設定する - ログを出力する - ローカルストレージにアクセスする - 外部サービスに接続する

これらの処理は `useEffect` フックの中で行います。

6.2 基本的な使い方

```
// =====
// useEffect の超基本サンプル（詳細コメント版）
// -----
// 「count が変わるたびに、ブラウザタブのタイトルも追従して変わる」コンポーネント
// =====

// useState（状態を持つフック）と useEffect（副作用を扱うフック）を取り込む
import { useState, useEffect } from "react";

function PageTitle() {
  // count: 数値の状態。初期値は 0。
  const [count, setCount] = useState<number>(0);

  // -----
  // useEffect: 「画面が描かれた *あと* に何かをする」ためのフック
  // -----
  // 第1引数: 実行する関数（副作用本体）
  // 第2引数: 依存配列（この配列の中身が変わったときだけ第1引数が実行される）
  //
  // ここでは [count] を指定しているので「count が変わったら関数を実行」となる。
  // 初回マウント時にも1回実行される（このときも count = 0 → 0 という変化扱い）。
  useEffect(() => {
    // document.title はブラウザタブ上部の文字を変える命令（DOM操作 = 副作用）。
    // 副作用を JSX の中で直接書くと不安定なので、useEffect の中に隔離する。
    document.title = `カウント: ${count}`;

    // 開発者ツールの Console タブに出力する。動作確認に便利。
    console.log(`useEffect が実行されました (count = ${count})`);
  }, [count]); // ← count が変わるたびに上の関数が再実行される

  // 画面は「現在のカウント」と「+1ボタン」だけ。
  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

▼ ブラウザ画面（初期表示）：

タブタイトル: 「カウント: 0」

← `document.title` が変更された

```
┌   カウント: 0   ┐  
└   [ +1 ]       ┘
```

▼ Console タブ（開発者ツール）の出力:

```
useEffect が実行されました (count = 0)
```

▼ 「+1」ボタンを3回押した後:

タブタイトル: 「カウント: 3」

画面: カウント: 3

Console:

```
useEffect が実行されました (count = 0)  ← 初回マウント時  
useEffect が実行されました (count = 1)  ← 1回目クリック後  
useEffect が実行されました (count = 2)  ← 2回目クリック後  
useEffect が実行されました (count = 3)  ← 3回目クリック後
```

依存配列のキモ: もし `[count]` を `[]`（空配列）にすると、初回しか実行されないため、タブタイトルが「カウント: 0」のまま固まります。逆に第2引数を**完全に省略**すると毎回再描画ごとに実行され、無限ループになりがちです。「何が変わったらこの副作用を再実行したいか？」を意識して書くのが鉄則です。

6.3 依存配列

依存配列の指定方法によって、`useEffect` の実行タイミングが変わります。

```

import { useState, useEffect } from "react";

function EffectExamples() {
  const [count, setCount] = useState<number>(0);
  const [name, setName] = useState<string>("");

  // パターン1: 毎回実行（依存配列なし）
  useEffect(() => {
    console.log("毎回のレンダリング後に実行");
  }); // ← 第2引数を省略

  // パターン2: 初回のみ実行（空の依存配列）
  useEffect(() => {
    console.log("コンポーネントのマウント時に1回だけ実行");
    // API からデータを取得する処理などをここに書く
  }, []); // ← 空の配列

  // パターン3: 特定の値が変わったときに実行
  useEffect(() => {
    console.log(`count が変わりました: ${count}`);
  }, [count]); // ← count が変わるたびに実行

  // パターン4: 複数の依存値
  useEffect(() => {
    console.log(`count または name が変わりました: ${count}, ${name}`);
  }, [count, name]); // ← count または name が変わるたびに実行

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <input value={name} onChange={(e) => setName(e.target.value)} />
    </div>
  );
}

```

依存配列	実行タイミング	用途
省略	毎回のレンダリング後	ほとんど使わない
<code>[]</code> （空配列）	初回マウント時のみ	API 初期データ取得
<code>[count]</code>	<code>count</code> が変わった時	値の変化に応じた処理
<code>[count, name]</code>	いずれかが変わった時	複数の値に応じた処理

実践例: API からデータを取得

```
import { useState, useEffect } from "react";

type User = {
  id: number;
  name: string;
  email: string;
};

function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // API からユーザーデータを取得
    const fetchUsers = async () => {
      try {
        setLoading(true);
        const response = await fetch("https://jsonplaceholder.typicode.com/users");

        if (!response.ok) {
          throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data: User[] = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : "不明なエラー");
      } finally {
        setLoading(false);
      }
    };

    fetchUsers();
  }, []); // 空配列 → 初回マウント時に1回だけ実行

  if (loading) {
    return <p>読み込み中...</p>;
  }

  if (error) {
    return <p style={{ color: "red" }}>エラー: {error}</p>;
  }

  return (
    <div>
      <h2>ユーザー一覧</h2>
      <ul>
```

```

    {users.map((user) => (
      <li key={user.id}>
        {user.name} ({user.email})
      </li>
    ))}
  </ul>
</div>
);
}

```

この結果、画面にはまず「読み込み中...」と表示されます。

API からデータ取得が完了すると、「ユーザー一覧」という見出しと、ユーザーのリストが表示されます:

- Leanne Graham (Sincere@april.biz)
- Ervin Howell (Shanna@melissa.tv)
- ... (以下続く)

ネットワークエラーが発生した場合は、赤字で「エラー: Failed to fetch」等と表示されます。

6.4 クリーンアップ

`useEffect` の中で `return` した関数は、**クリーンアップ関数**として実行されます。コンポーネントがアンマウント（画面から消える）されるとき、または次の effect が実行される前に呼ばれます。

```

import { useState, useEffect } from "react";

function Timer() {
  const [seconds, setSeconds] = useState<number>(0);
  const [isRunning, setIsRunning] = useState<boolean>(false);

  useEffect(() => {
    if (!isRunning) return; // タイマーが停止中なら何もしない

    // 1秒ごとにカウントアップ
    const intervalId = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    // クリーンアップ: タイマーを解除する
    return () => {
      clearInterval(intervalId);
      console.log("タイマーをクリーンアップしました");
    };
  }, [isRunning]); // isRunning が変わるたびに再設定

  const handleReset = () => {
    setIsRunning(false);
    setSeconds(0);
  };

  return (
    <div>
      <h2>タイマー</h2>
      <p style={{ fontSize: "48px", fontFamily: "monospace" }}>
        {Math.floor(seconds / 60)
          .toString()
          .padStart(2, "0")}
        :{(seconds % 60).toString().padStart(2, "0")}
      </p>
      <button onClick={() => setIsRunning(true)} disabled={isRunning}>
        開始
      </button>
      <button onClick={() => setIsRunning(false)} disabled={!isRunning}>
        停止
      </button>
      <button onClick={handleReset}>リセット</button>
    </div>
  );
}

```

この結果、画面には大きなフォントで「00:00」と表示され、3つのボタンがあります。

【開始】を押すと、1秒ごとにカウントアップ: 「00:01」 → 「00:02」 → ...

【停止】を押すと、カウントが止まります。

【リセット】を押すと、「00:00」に戻り、停止します。

ウィンドウサイズの監視（クリーンアップの実用例）

```
import { useState, useEffect } from "react";

type WindowSize = {
  width: number;
  height: number;
};

function WindowSizeDisplay() {
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    // リサイズイベントのハンドラ
    const handleResize = () => {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    // イベントリスナーを登録
    window.addEventListener("resize", handleResize);

    // クリーンアップ: イベントリスナーを解除
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []); // 初回マウント時にのみ設定

  return (
    <div>
      <h2>ウィンドウサイズ</h2>
      <p>幅: {windowSize.width}px</p>
      <p>高さ: {windowSize.height}px</p>
    </div>
  );
}
```

この結果、画面には「幅: 1920px」「高さ: 1080px」のように現在のウィンドウサイズが表示されます。ブラウザのウィンドウをリサイズすると、数値がリアルタイムに変化します。

6.5 コンポーネントライフサイクルと useEffect

1 マウント（初回表示）

コンポーネント関数が実行される



JSX が返される



DOM に反映



useEffect が実行される（依存配列: []）



2 更新（state/props 変更時）

繰り返し

state または props が変更



コンポーネント関数が再実行



新しい JSX が返される



差分を検出して DOM を更新



前回の useEffect のクリーンアップ関数を実行



新しい useEffect が実行される（依存配列の値が変わった場合）



3 アンマウント（画面から消える）

コンポーネントが DOM から削除

↓

useEffect のクリーンアップ関数を実行

■ useEffect 実行 ■ クリーンアップ実行

7. カスタムフック

7.1 なぜカスタムフックを作るのか

複数のコンポーネントで同じロジックを使い回したい場合、カスタムフックを作ります。

カスタムフックなしの場合:

```
// ComponentA.tsx
function ComponentA() {
  const [windowSize, setWindowSize] = useState({ width: 0, height: 0 });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({ width: window.innerWidth, height: window.innerHeight });
    };
    handleResize();
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>幅: {windowSize.width}</p>;
}

// ComponentB.tsx - 全く同じロジックをコピペ...
function ComponentB() {
  const [windowSize, setWindowSize] = useState({ width: 0, height: 0 });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({ width: window.innerWidth, height: window.innerHeight });
    };
    handleResize();
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>高さ: {windowSize.height}</p>;
}
```

このように同じコードを何度も書くのは **DRY 原則**（Don't Repeat Yourself）に反します。

7.2 基本的な作り方

カスタムフックのルール: - 関数名は **use** で始める（必須。React がフックとして認識する） - 中で `useState`、`useEffect` などのフックを使う - 値や関数を返す

useWindowSize フック

```
import { useState, useEffect } from "react";

type WindowSize = {
  width: number;
  height: number;
};

// カスタムフック: ウィンドウサイズを返す
function useWindowSize(): WindowSize {
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return windowSize;
}

// 使い方: どのコンポーネントでも簡単に使える!
function Header() {
  const { width } = useWindowSize();
  return <header>{width > 768 ? "デスクトップメニュー" : "モバイルメニュー"}</header>;
}

function Footer() {
  const { width, height } = useWindowSize();
  return (
    <footer>
      画面サイズ: {width} x {height}
    </footer>
  );
}
```

Header コンポーネントでは、画面幅が768pxより大きい場合「デスクトップメニュー」、小さい場合「モバイルメニュー」と表示されます。ウィンドウをリサイズすると自動的に切り替わります。

useLocalStorage フック

```
import { useState, useEffect } from "react";

// ローカルストレージと同期する state を提供するカスタムフック
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void] {
  // 初期値: ローカルストレージに値があればそれを使う
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  // state が変わったらローカルストレージにも保存
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error("ローカルストレージへの保存に失敗:", error);
    }
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}

// 使い方
function Settings() {
  const [theme, setTheme] = useLocalStorage<string>("theme", "light");
  const [fontSize, setFontSize] = useLocalStorage<number>("fontSize", 16);

  return (
    <div>
      <h2>設定</h2>

      <div>
        <label>テーマ: </label>
        <select value={theme} onChange={(e) => setTheme(e.target.value)}>
          <option value="light">ライト</option>
          <option value="dark">ダーク</option>
        </select>
      </div>

      <div>
        <label>フォントサイズ: {fontSize}px</label>
        <input
          type="range"
          min={12}
        />
      </div>
    </div>
  );
}
```

```

      max={24}
      value={fontSize}
      onChange={(e) => setFontSize(Number(e.target.value))}
    />
  </div>

  <p style={{ fontSize: `${fontSize}px` }}>
    このテキストのフォントサイズが変わります。
  </p>
</div>
);
}

```

この結果、画面にはテーマ切り替えのセレクトボックスとフォントサイズのスライダーが表示されます。

フォントサイズを20pxに変更すると、プレビューテキストのサイズが大きくなります。

ページを再読み込みしても設定が保持されます（ローカルストレージに保存されているため）。

useToggle フック

```
import { useState, useCallback } from "react";

// true/false を切り替えるシンプルなカスタムフック
function useToggle(initialValue: boolean = false): [boolean, () => void] {
  const [value, setValue] = useState<boolean>(initialValue);

  const toggle = useCallback(() => {
    setValue((prev) => !prev);
  }, []);

  return [value, toggle];
}

// 使い方
function App() {
  const [isMenuOpen, toggleMenu] = useToggle(false);
  const [isDarkMode, toggleDarkMode] = useToggle(false);
  const [isModalOpen, toggleModal] = useToggle(false);

  return (
    <div style={{ backgroundColor: isDarkMode ? "#333" : "#fff", color: isDarkMode ? "#fff" : "#333" }}>
      <button onClick={toggleDarkMode}>
        {isDarkMode ? "ライトモード" : "ダークモード"}に切り替え
      </button>

      <button onClick={toggleMenu}>
        メニュー{isMenuOpen ? "を閉じる" : "を開く"}
      </button>

      {isMenuOpen && (
        <nav>
          <ul>
            <li>ホーム</li>
            <li>書籍一覧</li>
            <li>設定</li>
          </ul>
        </nav>
      )}

      <button onClick={toggleModal}>モーダルを開く</button>

      {isModalOpen && (
        <div style={{ border: "2px solid #ccc", padding: "16px", margin: "16px" }}>
          <h3>モーダルの内容</h3>
          <p>これはモーダルウィンドウです。</p>
          <button onClick={toggleModal}>閉じる</button>
        </div>
      )}
    </div>
  );
}
```

```

    </div>
  );
}

```

この結果、画面には3つのボタンが表示されます。

【ダークモードに切り替え】を押すと、背景が黒・文字が白に変わり、ボタンのテキストが「ライトモードに切り替え」に変わります。

【メニューを開く】を押すと、ナビゲーションリストが表示され、ボタンが「メニューを閉じる」に変わります。

7.3 useBooks フックの予告

後の章（第5章以降）で、書籍管理アプリのために以下のような `useBooks` カスタムフックを実装します。

```

// 予告: 後の章で実装するカスタムフック
type UseBooksReturn = {
  books: Book[];
  loading: boolean;
  error: string | null;
  addBook: (book: Omit<Book, "id">) => Promise<void>;
  updateBook: (id: number, updates: Partial<Book>) => Promise<void>;
  deleteBook: (id: number) => Promise<void>;
  searchBooks: (query: string) => void;
  filteredBooks: Book[];
};

function useBooks(): UseBooksReturn {
  // API との通信、state 管理、検索ロジックなどを
  // このフックに集約する予定
  // → 第5章「API連携」で詳しく実装します
}

// 使い方 (完成イメージ)
function BookListPage() {
  const { books, loading, error, deleteBook, searchBooks, filteredBooks } = useBooks();

  if (loading) return <p>読み込み中...</p>;
  if (error) return <p>エラー: {error}</p>;

  return (
    <div>
      <SearchBar onSearch={searchBooks} />
      <BookList books={filteredBooks} onDelete={deleteBook} />
    </div>
  );
}

```

このカスタムフックを使うことで、書籍データの取得・追加・更新・削除・検索といったすべてのロジックが1箇所にまとまり、コンポーネントは UI の描画に集中できるようになります。

8. よくあるミスと対処法

8.1 state の直接変更

最も多い間違い: state のオブジェクトや配列を直接変更してしまうこと。

```

import { useState } from "react";

type User = {
  name: string;
  age: number;
};

function UserEditor() {
  const [user, setUser] = useState<User>({ name: "田中", age: 25 });
  const [items, setItems] = useState<string[]>(["A", "B", "C"]);

  // ===== NG な例 =====

  const badUpdateName = () => {
    // NG: オブジェクトのプロパティを直接変更
    user.name = "鈴木";
    setUser(user); // 同じ参照のオブジェクトなので React は変化を検知できない!
  };

  const badAddItem = () => {
    // NG: 配列を直接変更
    items.push("D");
    setItems(items); // 同じ参照の配列なので React は変化を検知できない!
  };

  const badSortItems = () => {
    // NG: sort は元の配列を変更する破壊的メソッド
    items.sort();
    setItems(items);
  };

  // ===== OK な例 =====

  const goodUpdateName = () => {
    // OK: 新しいオブジェクトを作成
    setUser({ ...user, name: "鈴木" });
  };

  const goodAddItem = () => {
    // OK: 新しい配列を作成
    setItems([...items, "D"]);
  };

  const goodSortItems = () => {
    // OK: コピーしてからソート
    setItems([...items].sort());
  };

  return (

```

```
<div>
  <p>名前: {user.name}</p>
  <p>アイテム: {items.join(", ")}</p>
  <button onClick={goodUpdateName}>名前を変更</button>
  <button onClick={goodAddItem}>アイテム追加</button>
  <button onClick={goodSortItems}>ソート</button>
</div>
);
}
```

なぜ直接変更がダメなのか: React は state の更新を参照の比較 (===) で検出します。同じオブジェクト/配列への参照のままだと、中身が変わっていても「変化なし」と判断され、再レンダリングが発生しません。

8.2 useEffect の無限ループ

```
import { useState, useEffect } from "react";

function InfiniteLoopExample() {
  const [count, setCount] = useState<number>(0);
  const [data, setData] = useState<string[]>([]);

  // ===== NG: 無限ループ =====

  // パターン1: 依存配列を省略して state を更新
  useEffect(() => {
    setCount(count + 1); // state 更新 → 再レンダリング → useEffect 再実行 → state 更新 → ...
  }); // 依存配列がない！

  // パターン2: useEffect 内で毎回新しいオブジェクト/配列を state に設定
  useEffect(() => {
    setData(["A", "B", "C"]); // 毎回新しい配列オブジェクトが作られる → 再レンダリング → ...
  }); // 依存配列がない！

  // パターン3: 依存配列に毎回変わる値を入れる
  useEffect(() => {
    console.log("実行");
  }, [{ key: "value" }]); // オブジェクトリテラルは毎回新しい参照 → 毎回実行

  // ===== OK: 正しい使い方 =====

  // 修正1: 適切な依存配列を指定
  useEffect(() => {
    setCount((prev) => prev + 1); // 初回のみ実行
  }, []); // 空配列 → 初回マウント時のみ

  // 修正2: 条件付きで実行
  useEffect(() => {
    if (data.length === 0) {
      setData(["A", "B", "C"]); // data が空のときだけ設定
    }
  }, [data.length]);

  // 修正3: useMemo でオブジェクトの参照を安定させる
  // (useMemo については次章以降で詳しく解説します)

  return <p>カウント: {count}</p>;
}
```

無限ループを防ぐチェックリスト:

1. `useEffect` の依存配列は省略していないか？

2. `useEffect` 内で依存配列に含まれる state を無条件に更新していないか？
3. 依存配列にオブジェクトリテラルや配列リテラルを直接書いていないか？
4. `useEffect` 内の関数が毎回再生成されていないか？

8.3 key の重要性

`key` は React がリスト内の各要素を識別するために使います。正しい `key` を指定しないと、予期しないバグが発生します。

```

import { useState } from "react";

type Item = {
  id: number;
  text: string;
};

function KeyExample() {
  const [items, setItems] = useState<Item[]>([
    { id: 1, text: "アイテム1" },
    { id: 2, text: "アイテム2" },
    { id: 3, text: "アイテム3" },
  ]);

  // 先頭に追加する
  const addToTop = () => {
    const newItem: Item = {
      id: Date.now(),
      text: `アイテム${items.length + 1}`,
    };
    setItems([newItem, ...items]);
  };

  return (
    <div>
      <button onClick={addToTop}>先頭に追加</button>

      <h3>NG: index を key に使用</h3>
      {items.map((item, index) => (
        <div key={index}>
          <span>{item.text}</span>
          {/* input の値が正しい要素に紐づかない! */}
          <input type="text" placeholder="メモを入力" />
        </div>
      ))}

      <h3>OK: 一意な id を key に使用</h3>
      {items.map((item) => (
        <div key={item.id}>
          <span>{item.text}</span>
          <input type="text" placeholder="メモを入力" />
        </div>
      ))}
    </div>
  );
}

```

この結果を確認するには、以下の操作をしてください:

1. 「NG」セクションの各入力欄に「メモA」「メモB」「メモC」と入力する
2. 【先頭に追加】ボタンをクリックする
3. **NG の方**: 入力したメモの位置がずれる! 「メモA」が新しいアイテムの横に表示される
4. **OK の方**: 入力したメモは元のアイテムに正しく紐づいたまま

これは、**index** を key にすると、先頭に要素を挿入したとき全要素の index が変わり、React が要素の対応を正しく把握できなくなるためです。

key のベストプラクティス:

```
// ベスト: データベースの ID を使う
{items.map((item) => <Item key={item.id} />)}

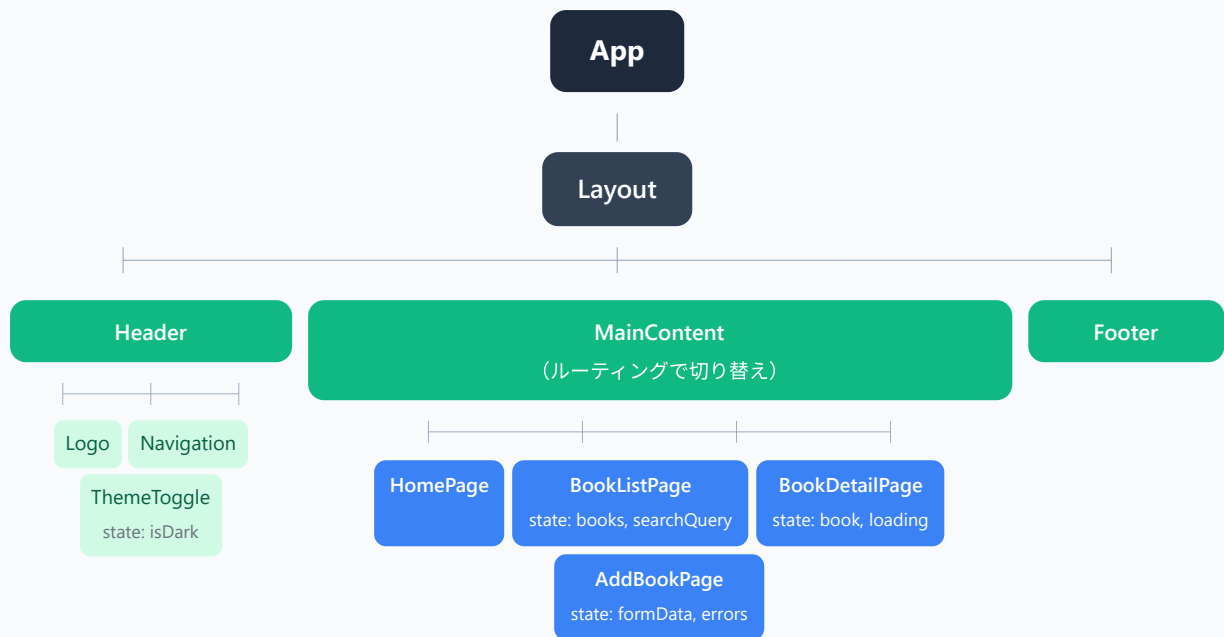
// OK: ユニークな文字列を使う
{items.map((item) => <Item key={item.slug} />)}

// 最終手段: index を使う (並び替え・追加・削除がない静的リストのみ)
{staticItems.map((item, index) => <Item key={index} />)}

// NG: ランダム値を使う (毎回変わるので意味がない)
{items.map((item) => <Item key={Math.random()} />)}
```

書籍管理アプリのコンポーネント設計

この章で学んだ知識を活かして、書籍管理アプリの全体像を確認しましょう。



ページ別の子コンポーネント

BookListPage の子コンポーネント

SearchBar

props: onSearch / state: query

FilterPanel

props: onFilter / state: selectedCategory

BookGrid

props: books

↳ BookGrid の子:

BookCard

props: book, onDelete, onToggleFavorite

BookCard

BookCard...

↳ BookCard の子:

BookImage

props: src, alt

BookInfo

props: title, author, price

BookActions

props: onEdit, onDelete

RatingStars

props: rating

AddBookPage の子コンポーネント

BookForm

props: onSubmit / state: formData, errors

↳ BookForm の子:

FormInput

props: label, value, onChange, error

FormSelect

props: options, value, onChange

Button

props: label, onClick, variant

BookDetailPage の子コンポーネント

BookDetail

props: book

ReviewList

props: reviews

↳ ReviewList の子:

ReviewCard

props: review

各コンポーネントにどの **state** が必要で、どの **props** を受け取るかが一目でわかります。後の章でこの設計に基づいて実装を進めていきます。

まとめ

この章で学んだ React の基礎をまとめます。

概念	要点
コンポーネント	UIの再利用可能な部品。関数として定義し、JSX を返す
JSX/TSX	JavaScript の中に HTML 風のコードを書ける構文
Props	親から子へデータを渡す仕組み。TypeScript で型を定義
State (useState)	コンポーネントが持つ変化するデータ。更新すると再レンダリング
イベントハンドリング	onClick, onChange, onSubmit でユーザー操作を処理
useEffect	副作用（API取得、DOM操作等）を実行するフック
カスタムフック	ロジックを再利用可能な関数に切り出す仕組み

次の章（第4章）では、React Router を使ったルーティングと、より実践的な画面遷移の実装に進みます。